



CropCare

Crop Disease Information Portal

An interactive web application enabling users to explore crops, search diseases, and access structured information on symptoms, prevention, and treatment — built as an academic software engineering project.

DOCUMENT TYPE	Software Requirements Specification (SRS)
STANDARD FOLLOWED	IEEE 830 / IEEE 29148 (adapted)
VERSION	1.0
DATE	August 14, 2026
PREPARED FOR	Faculty Evaluation / Academic Project Submission
STATUS	Final Draft — Ready for Review

Table of Contents

1. Introduction	3
1.1 Purpose	
1.2 Document Conventions	
1.3 Intended Audience and Reading Suggestions	
1.4 Project Scope	
1.5 Definitions, Acronyms and Abbreviations	
1.6 References	
2. Overall Description	5
2.1 Product Perspective	
2.2 Product Features (Summary)	
2.3 User Classes and Characteristics	
2.4 Operating Environment	
2.5 Design and Implementation Constraints	
2.6 Assumptions and Dependencies	
3. System Features & Functional Requirements	7
4. External Interface Requirements	11
5. System Architecture	13
6. Data Model / Database Design	14
7. Use Case Model	16
8. Non-Functional Requirements	18
9. Other Requirements	20
10. Appendices	21

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **CropCare - Crop Disease Information Portal**, a web-based application designed to help users explore crops and access structured information about the diseases that affect them, including symptoms, causes, prevention methods, and treatment options.

The purpose of this document is to provide a complete and unambiguous description of the system to be built, so that it can serve as a common reference point for the project team, the project guide/faculty evaluator, and any future developers who extend the system. This document defines *what* the system will do, not *how* it will be internally coded, and is intended to remain the single source of truth for requirement analysis, UI/UX design, database and API planning, software testing, and final project documentation.

1.2 Document Conventions

This document follows a structure adapted from the IEEE 830-1998 / IEEE 29148 SRS standards, simplified for an academic project of realistic scope. The following conventions are used throughout:

- Functional requirements are uniquely identified using the format `FR-XX`.
- Non-functional requirements are identified using the format `NFR-XX`.
- Priority levels are marked as **High**, **Medium**, or **Low**.
- Terms defined in Section 1.5 (Glossary) are used consistently across the document.
- `MUST` indicates a mandatory requirement; `SHOULD` indicates a recommended but non-mandatory requirement.

1.3 Intended Audience and Reading Suggestions

Audience	Recommended Sections	Purpose
Project Guide / Faculty Evaluator	Sections 1, 2, 3, 8	Evaluate scope, feasibility, and completeness
Student Developers (Frontend)	Sections 3, 4, 7	Understand UI screens and user interactions
Student Developers (Backend)	Sections 3, 5, 6	Understand APIs, architecture, and data model
Testers / QA Team Member	Sections 3, 8, 10	Design test cases and acceptance criteria
New Team Members	Whole document, start to end	Onboarding and full context

1.4 Project Scope

CropCare is an **information and awareness portal**, not a diagnostic or e-commerce system. The system allows a visitor to browse a curated catalogue of crops (e.g., Wheat, Rice, Cotton, Tomato, Sugarcane) and, for each crop, view the diseases known to affect it. Each disease entry presents structured information: description, symptoms, causes, prevention methods, and treatment/management recommendations, along with supporting images.

The system also includes a lightweight **Admin Panel** so that an authorized administrator (e.g., the project owner or a domain expert) can add, update, or remove crop and disease records without touching code — this keeps the content maintainable and demonstrates full CRUD capability for evaluation purposes.

The following are explicitly **within scope** for the initial version (V1) of the project:

- Browsing crops and their associated diseases in a structured, categorized layout.
- Keyword-based search for crops and diseases, with basic filters.
- Detailed disease information pages (symptoms, causes, prevention, treatment, images).
- A responsive, intuitive user interface usable on desktop and mobile browsers.
- An admin panel with secure login for content management (CRUD on crops and diseases).
- Basic contact/feedback form for site visitors.

The following are explicitly **out of scope** for V1 (documented here to keep the project realistic for a student team, and listed again in Section 9 as possible future enhancements):

- AI/ML-based image recognition for automatic disease detection from photographs.
- Real-time weather-based or satellite-based disease outbreak prediction.
- E-commerce features such as purchasing pesticides or fertilizers.
- Multi-language / regional language support (may be a future enhancement).
- Native mobile applications (Android/iOS) — the web app will be responsive instead.
- Community/forum features (user-to-user discussion, comments, ratings).

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
Crop	A cultivated plant record in the system (e.g., Wheat, Rice, Tomato)
Disease	A plant disease record associated with one or more crops
Admin	An authorized user who manages content through the Admin Panel
Visitor / End User	Any public user browsing the portal without logging in
CRUD	Create, Read, Update, Delete — basic data operations
API	Application Programming Interface
UI / UX	User Interface / User Experience
JWT	JSON Web Token, used for authentication sessions
Responsive Design	UI layout that adapts to different screen sizes (mobile, tablet, desktop)

1.6 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Requirements Engineering.
- Institutional guidelines for final-year / semester software engineering project submission.
- Publicly available agricultural extension resources on crop diseases (for reference content only, to be cited individually in the project report / data sources appendix).

2. Overall Description

2.1 Product Perspective

CropCare is a new, standalone, self-contained web application. It is not a replacement for or extension of any existing institutional system. It is developed independently as a three-tier web application consisting of a presentation layer (frontend), an application/business logic layer (backend API), and a data layer (database), described further in Section 5 (System Architecture).

Design Philosophy: The project intentionally favors a clean, well-structured, content-driven design over unnecessary complexity. The core value proposition is fast, reliable, and easy-to-understand access to crop disease information — not advanced analytics or automation.

2.2 Product Features (Summary)

The major feature groups of CropCare are summarized below; each is elaborated with full functional requirements in Section 3.

#	Feature Group	Description
1	Crop Catalogue	Browse all crops with images, names, and categories.
2	Disease Catalogue	Browse all diseases, filterable by crop or category.
3	Search	Search crops/diseases by keyword with autosuggestions.
4	Disease Detail View	Structured page: symptoms, causes, prevention, treatment, images.
5	Admin Content Management	Secure login; add/edit/delete crop and disease records.
6	Contact / Feedback	Simple form for visitors to send queries or feedback.
7	Responsive Navigation	Consistent, intuitive UI across devices.

2.3 User Classes and Characteristics

User Class	Characteristics	Privileges
Visitor / General User (farmers, students, agri-enthusiasts, general public)	No technical expertise assumed; may have varying levels of digital literacy; primary goal is quick, understandable information.	Browse, search, and view all public content. No login required.
Administrator (project owner / content manager)	Comfortable with basic web forms; responsible for keeping crop/disease data accurate and up to date.	Full CRUD access to crop and disease records via a protected Admin Panel.
Evaluator / Faculty	Reviews the system for correctness, completeness, and adherence to software engineering practices.	Same as Visitor, plus access to project documentation and (optionally) a demo admin account.

2.4 Operating Environment

- **Client side:** Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet, or mobile device with an internet connection.
- **Server side:** A backend server (e.g., Node.js/Express or Python/Django/Flask) hosted on a standard web server or a student-friendly cloud platform (e.g., Render, Railway, Vercel, or the college server).
- **Database:** A relational database (e.g., MySQL / PostgreSQL) or a document database (e.g., MongoDB), selected during the design phase (see Section 6).

2.5 Design and Implementation Constraints

- The project must be completable by a small student team (typically 2–4 members) within a single academic semester.
- The technology stack must consist of freely available, open-source tools with no licensing cost.
- The system must be deployable on free/low-cost student hosting tiers.
- Disease content must be factually sourced from credible agricultural references and clearly cited in project documentation (not a source-of-truth medical/legal claim).
- The UI must remain simple enough to be fully implemented and tested within the project timeline.

2.6 Assumptions and Dependencies

- Users have basic familiarity with using a web browser.
- The system assumes reasonably reliable internet connectivity for both users and the hosting environment.

- Initial crop and disease data will be manually curated and entered by the team/admin; no external live data feed is assumed for V1.
- Availability of royalty-free or self-created images for crops and diseases.

3. System Features & Functional Requirements

This section lists the detailed functional requirements of CropCare, grouped by feature area. Each requirement has a unique ID, priority, and a short description of expected system behavior, written to be directly usable for development tasks and test-case design.

3.1 Crop Browsing

FR-01 Display Crop List

Priority: High

The system **MUST** display a list/grid of all available crops on the Home or "Browse Crops" page, showing each crop's name, thumbnail image, and category (e.g., Cereal, Vegetable, Cash Crop).

FR-02 View Crop Detail Page

Priority: High

When a user selects a crop, the system **MUST** display a Crop Detail page containing the crop's description, image, category, and a list of diseases known to affect it.

FR-03 Filter Crops by Category

Priority: Medium

The system **SHOULD** allow users to filter the crop list by category (e.g., Cereals, Vegetables, Fruits, Cash Crops).

3.2 Disease Browsing & Detail

FR-04 Display Disease List per Crop

Priority: High

On a Crop Detail page, the system **MUST** list all diseases associated with that crop, each shown with a name, thumbnail image, and a one-line summary.

FR-05 View Disease Detail Page

Priority: High

The system **MUST** provide a Disease Detail page presenting, at minimum: disease name, affected crop(s), description, symptoms (list), causes/pathogen type, prevention methods (list), and treatment/management recommendations (list), along with at least one supporting image.

FR-06 Browse All Diseases

Priority: Medium

The system **SHOULD** provide a standalone "All Diseases" page listing every disease in the system, independent of navigating through a specific crop, to support users who know the disease name but not the crop.

3.3 Search

FR-07 Keyword Search

Priority: High

The system **MUST** provide a search bar (accessible from all major pages) that allows users to search by crop name or disease name using keyword matching.

FR-08 Search Results Page

Priority: High

The system **MUST** display search results grouped or labeled by type (Crop / Disease), each item linking to its respective detail page. If no results are found, an appropriate "no results" message **MUST** be shown.

FR-09 Search Autosuggestions

Priority: Low

The system **SHOULD** show live autosuggestions as the user types in the search bar, to improve discoverability.

3.4 Admin Content Management

FR-10 Admin Login

Priority: High

The system MUST provide a secure login page for administrators using a username/email and password. Access to the Admin Panel MUST NOT be possible without successful authentication.

FR-11 Manage Crops (CRUD)

Priority: High

An authenticated administrator MUST be able to Create, Read, Update, and Delete crop records, including name, category, description, and image.

FR-12 Manage Diseases (CRUD)

Priority: High

An authenticated administrator MUST be able to Create, Read, Update, and Delete disease records, including name, associated crop(s), description, symptoms, causes, prevention, treatment, and image(s).

FR-13 Admin Logout

Priority: Medium

The system MUST allow an authenticated administrator to log out, ending the active session.

3.5 Feedback / Contact

FR-14 Contact / Feedback Form

Priority: Low

The system SHOULD provide a contact form where visitors can submit their name, email, and a message (e.g., feedback, a correction, or a question). Submitted messages are stored for admin review.

3.6 Navigation & General

FR-15 Responsive Navigation Bar

Priority: High

The system MUST provide a consistent navigation bar/menu across all pages, giving access to Home, Browse Crops, Browse Diseases, Search, Contact, and (for admins) the Admin Panel.

FR-16 404 / Error Handling

Priority: Medium

The system MUST display a user-friendly error page for invalid routes (e.g., non-existent crop/disease IDs) instead of a raw technical error.

4. External Interface Requirements

4.1 User Interfaces

The user interface will follow a clean, card-based, content-first design using a natural green and yellow color palette (evoking crops and agriculture), with clear typography and iconography. Key screens include:

Screen	Purpose	Key Elements
Home Page	Landing page introducing CropCare	Hero banner, search bar, featured crops, quick links
Browse Crops	Grid/list of all crops	Crop cards with image, name, category filter
Crop Detail	Info about one crop + its diseases	Crop info panel, disease card list
Browse Diseases	Grid/list of all diseases	Disease cards, filter by crop/category
Disease Detail	Full disease information	Tabs/sections: Symptoms, Causes, Prevention, Treatment, Images
Search Results	Search output	Grouped results (crops/diseases), empty-state message
Contact Page	Feedback submission	Form with name, email, message, submit button
Admin Login	Authentication	Username/email + password fields, error messages
Admin Dashboard	Content management	Tables/lists of crops & diseases with Add/Edit/Delete actions

4.2 Hardware Interfaces

No special hardware is required. The system runs on standard client devices (desktop, laptop, tablet, smartphone) capable of running a modern web browser, and on standard server hardware/cloud infrastructure for hosting.

4.3 Software Interfaces

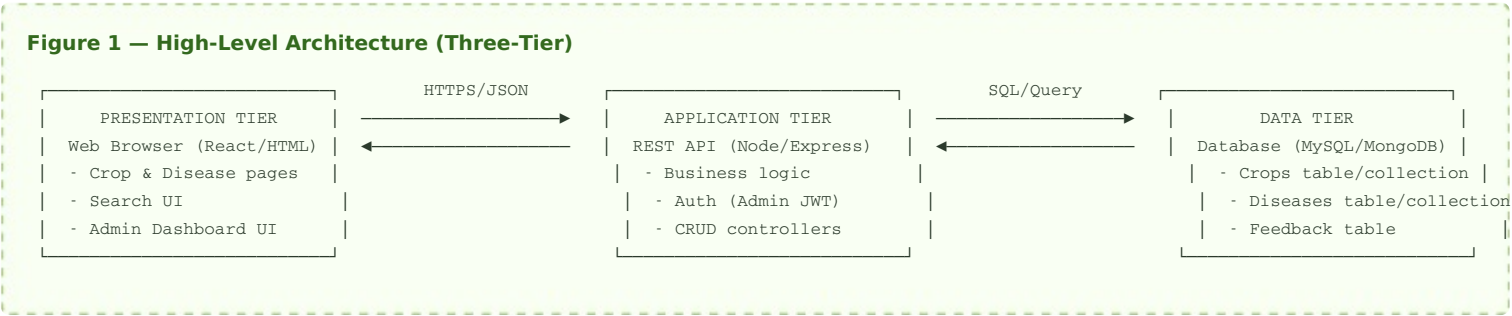
Layer	Suggested Technology (indicative — finalized in design phase)
Frontend	HTML5, CSS3, JavaScript (React.js or plain JS)
Backend	Node.js with Express.js (or Python with Flask/Django)
Database	MySQL / PostgreSQL (relational) or MongoDB (document-based)
Authentication	Session-based auth or JWT for the Admin Panel
Image Storage	Server file system / cloud storage (e.g., Cloudinary) for crop & disease images

4.4 Communication Interfaces

The frontend communicates with the backend over HTTPS using a RESTful API exchanging data in JSON format. Standard HTTP status codes (200, 201, 400, 401, 404, 500) MUST be used consistently across all API responses.

5. System Architecture

CropCare follows a standard **three-tier web architecture**, chosen for its simplicity, clear separation of concerns, and suitability for a student project timeline.



5.1 Component Overview

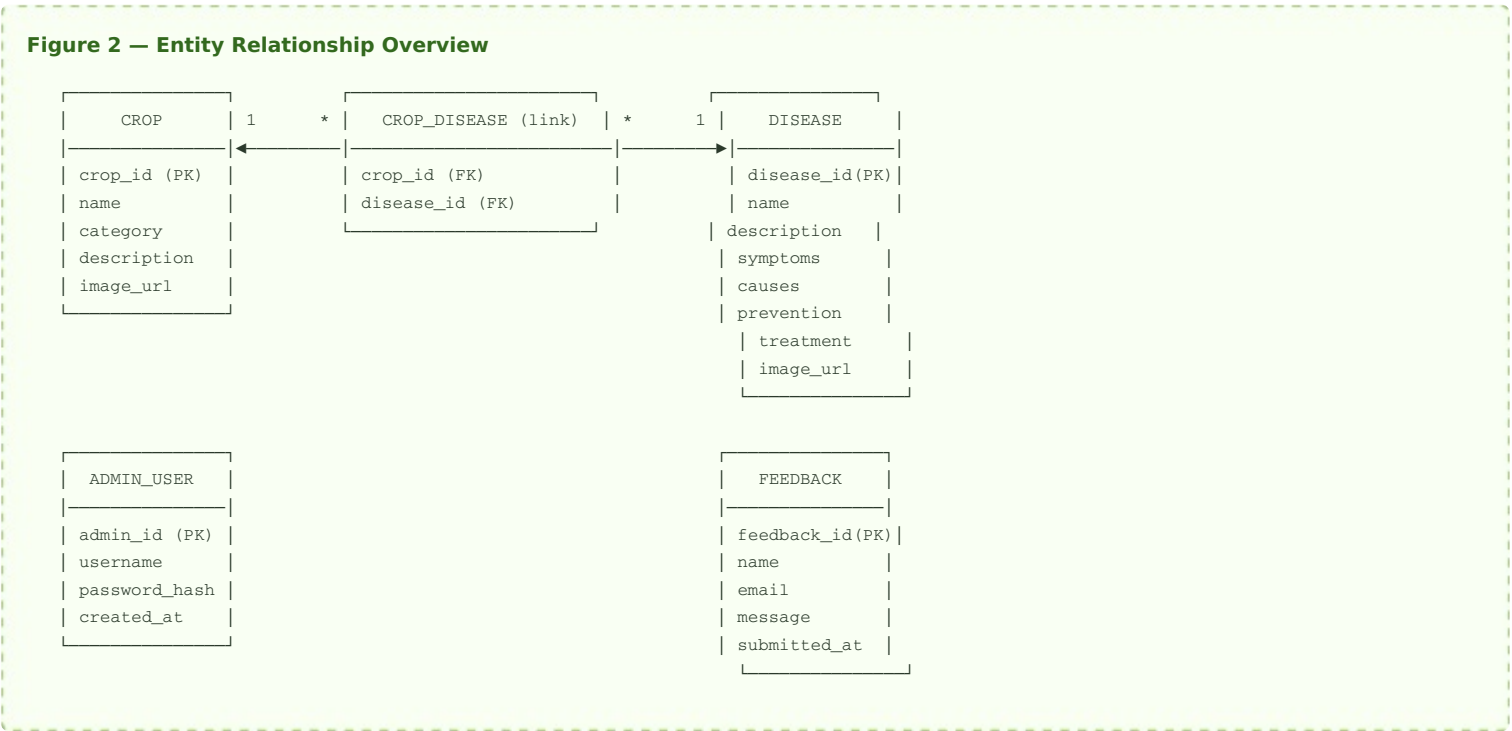
- **Presentation Tier:** Renders pages, handles client-side routing, calls backend APIs, and displays responses to the user.
- **Application Tier:** Exposes REST endpoints, implements business logic (e.g., search filtering, validation), enforces admin authentication/authorization, and talks to the database.
- **Data Tier:** Persistently stores crop, disease, and feedback data.

5.2 Indicative REST API Endpoints

Method	Endpoint	Description	Access
GET	/api/crops	List all crops (supports category filter)	Public
GET	/api/crops/:id	Get one crop with its diseases	Public
GET	/api/diseases	List all diseases (supports crop filter)	Public
GET	/api/diseases/:id	Get full detail of one disease	Public
GET	/api/search?q=	Search crops & diseases by keyword	Public
POST	/api/auth/login	Admin login, returns session/JWT token	Public (credentials required)
POST	/api/crops	Create a new crop	Admin only
PUT	/api/crops/:id	Update a crop	Admin only
DELETE	/api/crops/:id	Delete a crop	Admin only
POST	/api/diseases	Create a new disease	Admin only
PUT	/api/diseases/:id	Update a disease	Admin only
DELETE	/api/diseases/:id	Delete a disease	Admin only
POST	/api/feedback	Submit a contact/feedback message	Public

6. Data Model / Database Design

The core data model is intentionally simple: a many-to-many relationship between **Crops** and **Diseases** (one disease may affect multiple crops; one crop may be affected by multiple diseases), plus supporting entities for images, admin users, and feedback.



6.1 Table Definitions

Crop

Field	Type	Constraint	Description
crop_id	Integer	Primary Key, Auto-increment	Unique crop identifier
name	Varchar(100)	Not Null, Unique	Crop name (e.g., "Wheat")
category	Varchar(50)	Not Null	e.g., Cereal, Vegetable, Cash Crop
description	Text	Not Null	Short description of the crop
image_url	Varchar(255)	Nullable	Path/URL of the crop's image

Disease

Field	Type	Constraint	Description
disease_id	Integer	Primary Key, Auto-increment	Unique disease identifier
name	Varchar(120)	Not Null	Disease name (e.g., "Leaf Rust")
description	Text	Not Null	General overview of the disease
symptoms	Text	Not Null	List of observable symptoms
causes	Text	Nullable	Pathogen / cause type (fungal, bacterial, viral, pest)
prevention	Text	Not Null	Preventive measures
treatment	Text	Not Null	Recommended treatment/management steps
image_url	Varchar(255)	Nullable	Path/URL of a representative image

Crop_Disease (Junction Table)

Field	Type	Constraint	Description
crop_id	Integer	Foreign Key → Crop.crop_id	Linked crop
disease_id	Integer	Foreign Key → Disease.disease_id	Linked disease

Admin_User

Field	Type	Constraint	Description
admin_id	Integer	Primary Key, Auto-increment	Unique admin identifier

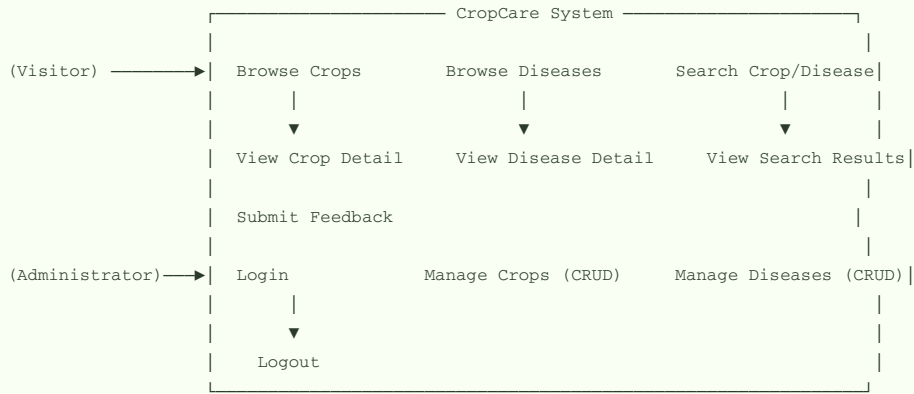
username	Varchar(50)	Not Null, Unique	Login username
password_hash	Varchar(255)	Not Null	Securely hashed password
created_at	Timestamp	Not Null, Default: Now	Account creation time

Feedback

Field	Type	Constraint	Description
feedback_id	Integer	Primary Key, Auto-increment	Unique feedback identifier
name	Varchar(100)	Not Null	Sender's name
email	Varchar(100)	Not Null	Sender's email
message	Text	Not Null	Feedback / query content
submitted_at	Timestamp	Not Null, Default: Now	Submission time

7. Use Case Model

Figure 3 — Use Case Diagram (Textual Representation)



7.1 Use Case: Search for a Disease

Field	Description
Use Case ID	UC-01
Actor	Visitor
Precondition	User is on any page of CropCare
Main Flow	1. User enters a keyword in the search bar. 2. System queries crops and diseases matching the keyword. 3. System displays grouped results (crops / diseases). 4. User selects a result to view its detail page.
Alternate Flow	If no matches are found, system displays a "No results found" message with a suggestion to browse categories.
Postcondition	User is viewing either search results or a detail page.

7.2 Use Case: Manage Disease Record (Admin)

Field	Description
Use Case ID	UC-02
Actor	Administrator
Precondition	Admin is authenticated and on the Admin Dashboard
Main Flow	1. Admin selects "Add Disease" or selects an existing disease to edit. 2. Admin fills/updates the form (name, crop association, symptoms, causes, prevention, treatment, image). 3. Admin submits the form. 4. System validates input and saves the record. 5. System displays a success confirmation and updated list.
Alternate Flow	If validation fails (e.g., missing required field), system highlights the error and prevents submission.
Postcondition	Disease record is created/updated and immediately visible to visitors.

8. Non-Functional Requirements

8.1 Performance Requirements

ID	Requirement
NFR-01	Standard pages (crop list, disease detail) should load within 2–3 seconds under normal conditions on a typical broadband/mobile-data connection.
NFR-02	Search results should be returned within 2 seconds for a database of up to a few hundred records (realistic scope for this project).

8.2 Security Requirements

ID	Requirement
NFR-03	Admin passwords MUST be stored using a secure hashing algorithm (e.g., bcrypt); plain-text passwords MUST NOT be stored.
NFR-04	Admin-only API endpoints MUST verify a valid authentication token/session before allowing create, update, or delete operations.
NFR-05	All user inputs (search queries, contact form, admin forms) MUST be validated and sanitized on both client and server side to prevent injection attacks.

8.3 Usability Requirements

ID	Requirement
NFR-06	The interface MUST be intuitive enough that a first-time visitor can find a specific disease's prevention method within 3 clicks from the home page.
NFR-07	The UI MUST be responsive and usable on screen widths from 360px (mobile) to 1920px (desktop).
NFR-08	Text content MUST use plain, non-technical language wherever possible, given the general-public target audience.

8.4 Reliability & Availability

ID	Requirement
NFR-09	The system should handle invalid or unexpected inputs gracefully, without crashing, and display appropriate error messages.
NFR-10	The public-facing site should target at least 95% uptime during the evaluation/demo period, subject to hosting-provider limitations typical of free student tiers.

8.5 Maintainability & Portability

ID	Requirement
NFR-11	The codebase MUST follow a modular structure (separate folders for routes, controllers, models, and UI components) to support maintainability and team collaboration.
NFR-12	The application MUST be deployable to any standard Node.js/Python-compatible hosting environment without platform-specific code.

8.6 Software Quality Attributes Summary

- Usability
- Performance
- Security
- Maintainability
- Portability
- Reliability
- Scalability (basic)

9. Other Requirements

9.1 Legal & Content Accuracy

All agricultural information presented (symptoms, prevention, treatment) is for **informational and educational purposes only** and must be clearly labeled as such. The application is not a substitute for professional agronomic or agricultural-extension advice, and this disclaimer should be visible in the site footer.

9.2 Future Enhancements (Out of Current Scope)

Enhancement	Rationale for Deferral
AI-based disease detection from uploaded leaf images	Requires ML model training/dataset; significant added complexity
Multi-language support	Adds translation/content-management overhead beyond project timeline
Weather-based outbreak alerts	Requires third-party weather API integration and predictive logic
User accounts for visitors (favorites, history)	Not essential to core information-delivery goal of V1
Community discussion / comments	Introduces moderation and abuse-prevention requirements

10. Appendices

Appendix A — Sample Crop & Disease Data (Illustrative)

Crop	Category	Sample Diseases
Wheat	Cereal	Leaf Rust, Powdery Mildew, Loose Smut
Rice	Cereal	Blast Disease, Bacterial Leaf Blight, Sheath Blight
Tomato	Vegetable	Early Blight, Late Blight, Leaf Curl Virus
Cotton	Cash Crop	Bacterial Blight, Root Rot, Wilt
Sugarcane	Cash Crop	Red Rot, Smut, Rust

Appendix B — Traceability Notes

Each functional requirement (FR-XX) in Section 3 maps directly to at least one UI screen in Section 4.1 and, where applicable, to one or more API endpoints in Section 5.2. This traceability should be preserved in the test plan, where every FR is covered by at least one test case.

Appendix C — Document Approval

Role	Name	Signature / Date
Prepared By (Student Team)		
Reviewed By (Project Guide)		
Approved By (Faculty Evaluator)		